เอกสารสรุปโครงการ

CS2 Stats Tracker

Alpha Version | ใช้ข้อมูลจาก Leetify API

เอกสารฉบับนี้สรุปความสามารถ โครงสร้าง
และข้อจำกัดที่ตรวจพบจากซอร์สโค้ดในโปรเจกต์
เพื่อใช้ประกอบการนำเสนอและวางแผนพัฒนารุ่นถัดไป

จัดทำจากไฟล์ app.py, leetify_client.py, main.py, templates/ และ requirements.txt
สถานะข้อมูล: อ้างอิงการตรวจไฟล์ซอร์ส ไม่ใช่ผลทดสอบการเชื่อมต่อบริการภายนอก

1. ภาพรวมและเป้าหมาย

CS2 Stats Tracker เป็นแอปพลิเคชันสำหรับค้นหาและแสดงสถิติผู้เล่น Counter-Strike 2 จากบริการ Leetify โดยมีทั้งเว็บแอป Flask และโปรแกรม command line ที่เรียกใช้ client ชุดเดียวกัน

เป้าหมายที่เห็นจากโค้ด

- ค้นหาผู้เล่นจาก Steam ID หรือข้อมูลโปรไฟล์ Steam ที่รองรับ
- แสดงภาพรวมโปรไฟล์และคะแนนทักษะในหน้าที่อ่านง่าย
- ดูผลการแข่งขันล่าสุดและเปรียบเทียบผู้เล่นสองคน
- จัดรูปแบบข้อมูลจาก Leetify ให้อยู่ในโครงสร้างกลางเพื่อใช้ซ้ำในหน้าเว็บและ CLI

ผู้ใช้เป้าหมายที่สอดคล้องกับฟีเจอร์

ผู้เล่น CS2 ที่ต้องการดูภาพรวมผลงานตนเอง หรือเปรียบเทียบสถิติระหว่างผู้เล่นสองคน
ทั้งนี้ไฟล์โครงการยังไม่ได้ระบุกลุ่มผู้ใช้หรือ use case อย่างเป็นทางการ

องค์ประกอบของโครงการ

องค์ประกอบ	บทบาท
app.py	กำหนดหน้าเว็บ Flask, ประมวลผลแบบฟอร์ม, เปรียบเทียบผู้เล่น และให้ JSON endpoint
leetify_client.py	แปลงตัวระบุ Steam, เรียก Leetify API, จัดรูปแบบข้อมูลผู้เล่นและแมตช์
main.py	เมนู command line สำหรับโปรไฟล์ แมตช์ และเปรียบเทียบ
templates/ และ static/	HTML templates และ CSS สำหรับหน้าจอเว็บ
requirements.txt	ระบุ Flask, requests และ python-dotenv

ขอบเขตเอกสาร: สรุปสิ่งที่โค้ดแสดงว่ามีอยู่แล้ว ส่วน roadmap, SLA, rate limit
และผลการทดสอบจริงไม่มีระบุในไฟล์ที่ตรวจสอบ

2. ขอบเขตฟีเจอร์ Alpha

ฟีเจอร์ต่อไปนี้มีหลักฐานอยู่ใน route, client function และ HTML template ของโปรเจกต์

ฟีเจอร์	สิ่งที่ผู้ใช้ทำได้	ข้อมูลที่แสดง/กติกา
โปรไฟล์	ส่ง Steam ID หรือ URL เพื่อค้นหาผู้เล่น	ชื่อ, Steam64 ID, จำนวนแมตช์, win rate, Premier, FACEIT และคะแนน 4 หมวด
ประวัติแมตช์	กรอก ID และจำนวนแมตช์ที่ต้องการ	แผนที่, ชนะ/แพ้, สกอร์สองทีม และวันที่/เวลา; เว็บ clamp จำนวน 1-30
เปรียบเทียบ	กรอกตัวระบุผู้เล่นสองคน	วางสถิติผู้เล่นสองคนเทียบกัน และไฮไลต์คะแนนทักษะที่สูงกว่า
JSON profile	เรียก GET /api/player/	คืนข้อมูลโปรไฟล์ JSON หรือ HTTP 404 เมื่อไม่มีข้อมูล

รูปแบบตัวระบุผู้เล่น

- Steam Community profile URL ที่มี Steam64 ID
- Steam vanity URL รูปแบบ /id/ และ vanity name ที่เป็นตัวอักษร ตัวเลข ขีดล่าง หรือขีดกลาง โดย resolve ผ่าน Steam Community XML
- SteamID แบบ STEAM_X:Y:Z, SteamID3 แบบ [U:1:...], Steam64 17 หลัก และ account ID ตัวเลขล้น

รายละเอียดการเปรียบเทียบ

โค้ดเปรียบเทียบคะแนน Aim, Positioning, Utility และ Clutch ทีละหมวด หากทั้งสองคะแนนมีค่า truthy จะตัดสินผู้ที่มีคะแนนสูงกว่า หรือแสดงว่าเท่ากัน หากคะแนนขาดหรือเป็นศูนย์ แสดงว่าไม่มีข้อมูลในผลเปรียบเทียบ

จุดที่ควรอธิบายต่อผู้ฟัง

โปรไฟล์และประวัติแมตช์มาจากข้อมูลที่ Leetify ส่งกลับ
การแสดงผลขึ้นกับการมีข้อมูลของผู้เล่นและความสำเร็จในการเรียกบริการภายนอก

3. การทำงานและการเชื่อมต่อ API

ลำดับการทำงานโดยย่อ

ขั้น	การทำงาน
1. รับข้อมูลค้นหา	ผู้ใช้ส่ง Steam ID, vanity name หรือลิงก์จากหน้าเว็บหรือ CLI
2. Resolve Steam ID	resolve_steam64 แปลงอินพุตให้เป็น Steam64; vanity อาศัย Steam Community XML
3. เรียก Leetify	ส่ง GET ไปยัง public API profile หรือ matches พร้อม steam64_id
4. Normalize	แปลงชื่อฟิลด์และค่าตัวเลขให้อยู่ในโครงสร้างมาตรฐานของแอป
5. แสดงผล	เว็บ render HTML, CLI พิมพ์ข้อความ หรือ JSON route ส่งข้อมูลโปรไฟล์

เส้นทาง API ที่พบ

ระบบ	Method / path	การใช้งาน
แอป Flask	GET /	หน้าแรก
แอป Flask	GET, POST /profile	แสดง/ค้นหาโปรไฟล์ผ่านแบบฟอร์ม
แอป Flask	GET, POST /matches	แสดง/ค้นหาประวัติแมตช์
แอป Flask	GET, POST /compare	เปรียบเทียบผู้เล่นสองคน
แอป Flask	GET /api/player/	ส่ง JSON โปรไฟล์; 404 เมื่อไม่พบ
Leetify	GET /v3/profile	ดึงโปรไฟล์ด้วย steam64_id
Leetify	GET /v3/profile/matches	ดึงรายการแมตช์ด้วย steam64_id

การยืนยันตัวตนและ timeout

โค้ดอ่าน LEETIFY_API_KEY จาก environment ผ่าน python-dotenv และส่งใน header ชื่อ _leetify_key เมื่อมีการตั้งค่าโปรไฟล์เรียก API มี timeout 20 วินาที; matches 30 วินาที; การ resolve vanity ผ่าน Steam Community มี timeout 15 วินาที

ไม่มีการฝังค่า key ลงใน source code ที่ตรวจสอบ เอกสารนี้จงใจไม่บรรจุค่า secret จากไฟล์ตั้งค่า

4. รูปแบบข้อมูลทีแอปใช้

ข้อมูลโปรไฟล์ (หลัง normalize)

ฟิลด์	ความหมาย/ทีมาในแอป
name	ชื่อผู้เล่นจาก profile
steam64_id	Steam64 ID ที resolve แล้ว
totalMatches	จำนวนแมตช์; รองรับชื่อฟิลด์ต้นทาง total_matches หรือ totalMatches
winRate	อัตราชนะเป็นเปอร์เซ็นต์; หากต้นทางส่งสัดส่วนไม่เกิน 1 จะคูณ 100
ranks.premier / ranks.faceit	อันดับ Premier และระดับ FACEIT ถ้ามี
ratings.aim / positioning / utility / clutch	คะแนนทักษะ ปัดทศนิยมสองตำแหน่ง
rating_bars.*	ค่าที clamp ให้อยู่ในช่วง 0-100 สำหรับแถบแสดงผล
raw	ข้อมูลดิบทั้งก้อนจาก Leetify ทีเก็บไว้ในผล player

ข้อมูลแมตช์ (หลัง normalize)

ฟิลด์	ความหมาย/วิธีคำนวณ
mapName	ชื่อแผนที หรือ Unknown หากไม่มี
won	ผลชนะจาก rounds_won > rounds_lost ของผู้เล่น; ใช้ outcome/won เป็น fallback
teamScores	สกอร์สองทีมจาก team_scores, score หรือ teamScores
startedAt	เลือก finished_at, startedAt หรือ started_at ตามลำดับ

ข้อสังเกตด้านความหมายข้อมูล

รูปแบบและความหมายของข้อมูลต้นทางขึนกับ Leetify API การ normalize พยายามรองรับชื่อฟิลด์หลายรูปแบบ แต่ยังไม่มี schema validation หรือเอกสาร data dictionary ของ API ใน repository

5. วิธีใช้งานและสาธิต

การติดตั้งตามส่วนประกอบที่พบ

requirements.txt ระบุแพ็กเกจ Flask, requests และ python-dotenv; โค้ด client โหลดตัวแปรจากไฟล์ .env

ขั้น	รายละเอียด
1	ติดตั้ง dependencies ที่ระบุใน requirements.txt
2	ตั้งค่า LEETIFY_API_KEY ใน .env โดยไม่เผยแพร่ key
3	เริ่มเว็บผ่าน app.py; หน้าแรกมีทางเข้าโปรไฟล์ แมตช์ และเปรียบเทียบ
4	หรือรัน main.py เพื่อใช้เมนู CLI

แนวทางสาธิต 3-5 นาที

- ค้นหาโปรไฟล์จาก Steam ID แล้วชี้ให้เห็นจำนวนแมตช์ win rate, ranks และคะแนนทักษะ
- เปิดประวัติแมตช์และอธิบาย map, ผล, score, เวลา พร้อมตัวเลือกจำนวนแมตช์
- เปรียบเทียบผู้เล่นสองคนเพื่อให้เห็นรูปแบบ side-by-side และคะแนนเด่นรายหมวด
- สาธิต endpoint JSON /api/player/ หากผู้ฟังเป็นกลุ่มเทคนิค

ตัวอย่างโครงสร้าง JSON โดยย่อ

{ "name": "...", "steam64_id": "...", "totalMatches": 0, "winRate": 0.0, "ranks": {"premier": null, "faceit": null}, "ratings": {"aim": 0, "positioning": 0, "utility": 0, "clutch": 0}, "rating_bars": {"...": 0}, "raw": {} }

ตัวอย่างข้างต้นเป็น schema สรุปจากโค้ด ไม่ใช่ข้อมูลจริงจากผู้เล่น และไม่ควรใช้ค่า 0 เป็นตัวแทนข้อมูลที่ขาดโดยไม่อธิบาย

6. ข้อจำกัด ความเสี่ยง และประเด็นติดตาม

สิ่งที่พบในโค้ด	ผลต่อการใช้งาน/สิ่งที่ควรสื่อสาร
ความผิดพลาดเครือข่ายหรือ HTTP status ที่ไม่ใช่ 200 มักคืน None หรือ []	ผู้ใช้อาจเห็นเป็นไม่พบผู้เล่น/ไม่มีแมตช์ แยก API ล่ม, auth ไม่ผ่าน, ID ผิด หรือไม่มีข้อมูลไม่ชัด
ข้อมูลต้นทางไม่ถูกตรวจด้วย schema	การเปลี่ยนชื่อฟิลด์หรือชนิดข้อมูลจากผู้ให้บริการอาจทำให้แสดงผลผิดหรือว่าง
มี raw response อยู่ใน player object	JSON route ส่งข้อมูล profile object ทั้งชุด รวม raw; ควรตรวจว่าข้อมูลที่ส่งออกเหมาะสมก่อนเปิดให้ผู้ใช้ทั่วไป
app.py เปิด debug=True เมื่อรันตรง	ควรปิด debug ในการ deploy และกำหนดการรัน production ให้ชัด
CLI แปลงจำนวนแมตช์ด้วย int โดยตรง	อินพุตที่ไม่ใช่ตัวเลขอาจทำให้โปรแกรมจบด้วย exception
requirements.txt ระบุชื่อแพ็กเกจ แต่ไม่ pin เวอร์ชัน	ติดตั้งซ้ำอาจได้ dependency คนละเวอร์ชัน

สิ่งที่ยังยืนยันไม่ได้จากไฟล์

- การเชื่อมต่อและการยืนยัน key ใช้งานได้จริง (การทดสอบจากสภาพแวดล้อมนี้ติดปัญหาเครือข่าย)
- อัตราการเรียกสูงสุดหรือ rate limit, SLA, เงื่อนไขการใช้ข้อมูล และความเสถียรของ public API
- ความถูกต้องของสูตรคะแนนหรือความหมายเชิงลึกของ rating จาก Leetify
- มี test suite, CI/CD, logging, caching หรือ production deployment หรือไม่

ข้อเสนอสำหรับเวอร์ชันถัดไป (เป็นข้อเสนอ ไม่ใช่ฟีเจอร์ที่มีในปัจจุบัน)

- แยกสถานะ error เป็น invalid input, not found, unauthorized, rate limited และ upstream unavailable
- เพิ่ม input validation, schema checks, structured logging และ test cases
- เพิ่ม cache / retry แบบมีขอบเขตตามข้อกำหนดของ Leetify
- ย้าย debug และ secret handling ไปสู่ configuration สำหรับ environment
- บันทึกแหล่งที่มาของข้อมูลและเวลาที่ดึง เพื่ออธิบายความสดใหม่ของสถิติ

7. สรุปสำหรับการนำเสนอ

ข้อความสรุป: Alpha นีพิสูจน์แนวทางการดึงข้อมูล Leetify มาจัดรูปแบบ แล้วนำเสนอเป็นหน้าโปรไฟล์ ประวัติแมตช์ และการเปรียบเทียบผู้เล่น ผ่านเว็บและ command line

ประเด็นที่ควรจำ

- ฟังก์ชันหลักใช้งานผ่านข้อมูล Steam ID และอาศัย Leetify เป็นแหล่งข้อมูลภายนอก
- จุดเด่นของ Alpha คือ flow ใช้งานง่ายและมีทั้ง UI กับ endpoint JSON
- ก่อนขยายผู้ใช้ ควรทำ error handling, validation, security configuration และยืนยันการทำงานกับ API ใน environment ที่เชื่อมต่อได้
- ตัวเลขและตัวอย่างในสไลด์ควรใช้ข้อมูลทดสอบที่ได้รับอนุญาต และไม่เปิดเผย API key

ไฟล์อ้างอิงที่ตรวจสอบ

ไฟล์	ข้อมูลที่ใช้
app.py	Flask routes, form handling, compare logic, JSON endpoint
leetify_client.py	Steam ID resolution, Leetify requests, normalization
main.py	CLI flows และการแสดง profile/matches/compare
templates/*.html	เนื้อหาและฟิลด์ที่แสดงบนหน้าเว็บ
requirements.txt, .gitignore	dependencies และรูปแบบไฟล์ที่ ignore

เอกสารจัดทำจากซอร์สโค้ดใน workspace ณ วันที่ 30 กันยายน 2026